```
# Рисунки для статьи. Запускать ПОСЛЕ  %run vitaldb_aki.py
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.metrics import roc_auc_score

plt.rcParams.update({"font.size": 10, "font.family": "DejaVu Sans"})
COL = {"Park": "#1f77b4", "Nishimoto": "#ff7f0e",
       "Nepogodiev": "#2ca02c", "Bell": "#d62728"}
NAME = {"Park": "Park 2019 (SPARK)", "Nishimoto": "Nishimoto 2021",
        "Nepogodiev": "Nepogodiev 2018", "Bell": "Bell 2015"}
Bn = sm.families.Binomial()
y = c["aki"].astype(int).values
n = len(y)

# ---------- Figure 1: flow diagram ----------
N = dict(st)
steps = [
    ("All surgical cases in VitalDB", N["все операции"]),
    ("Adults (age ≥18 yr)", N["взрослые"]),
    ("Surgery duration ≥60 min", N["операция >= 60 мин"]),
    ("First surgery per patient", N["1 операция на пациента"]),
    ("Kidney/ureter surgery and transplantation excluded",
     N["без почек и трансплантаций"]),
    ("Complete predictor data", N["полные данные"]),
    ("eGFR ≥15 ml/min/1.73 m² — sensitivity cohort",
     N["СКФ >= 15 (анализ чувствит.)"]),
    ("Postoperative creatinine available — primary cohort",
     N["есть креатинин после (основной)"]),
]
excl = ["Age <18 yr", "Duration <60 min", "Repeat surgery",
        "Kidney/ureter or transplant", "Missing data",
        "eGFR <15", "No postoperative creatinine"]
fig, ax = plt.subplots(figsize=(8, 10))
ax.axis("off")
ys = np.linspace(0.95, 0.05, len(steps))
for i, (lab, k) in enumerate(steps):
    fc = "#dbe9f6" if i == len(steps) - 1 else "white"
    ax.text(0.33, ys[i], f"{lab}\nn = {k:,}", ha="center", va="center",
            fontsize=9.5, bbox=dict(boxstyle="round,pad=0.5",
                                    fc=fc, ec="black"))
    if i < len(steps) - 1:
```

```
        ax.annotate("", xy=(0.33, ys[i + 1] + 0.035),
                    xytext=(0.33, ys[i] - 0.035),
                    arrowprops=dict(arrowstyle="->"))
        mid = (ys[i] + ys[i + 1]) / 2
        ax.annotate("", xy=(0.66, mid), xytext=(0.33, mid),
                    arrowprops=dict(arrowstyle="->"))
        ax.text(0.68, mid, f"Excluded: {excl[i]}\nn = {k - steps[i + 1][1]:,}",
                va="center", fontsize=8.5,
                bbox=dict(boxstyle="round,pad=0.4", fc="#f2f2f2", ec="grey"))
ax.text(0.33, ys[-1] - 0.06,
        f"Postoperative AKI (KDIGO, 7 days): n = {y.sum()} ({y.mean():.1%})",
        ha="center", fontsize=9.5, style="italic")
ax.set_xlim(0, 1); ax.set_ylim(-0.05, 1)
plt.savefig("Figure1_flow.png", dpi=300, bbox_inches="tight")
plt.show()

# ---------- Figure 2: calibration with 95% CI ----------
def groups(p, k=10):
    g = pd.qcut(p, k, labels=False, duplicates="drop")
    t = pd.DataFrame({"g": g, "p": p, "y": y}).groupby("g")
    m = t.mean()
    cnt = t.size().values
    obs = m["y"].values
    se = np.sqrt(obs * (1 - obs) / cnt)
    return m["p"].values, obs, 1.96 * se

fig, axs = plt.subplots(2, 2, figsize=(9, 9))
for i, nm in enumerate(X):
    lp = X[nm].values
    p = 1 / (1 + np.exp(-lp))
    m1 = sm.GLM(y, sm.add_constant(lp), family=Bn).fit()
    p2 = m1.predict(sm.add_constant(lp))
    m0 = sm.GLM(y, np.ones(n), family=Bn, offset=lp).fit()
    a = axs.flat[i]
    for pp, lab, mk in [(p, "Original", "o"), (p2, "Recalibrated", "s")]:
        xp, ob, ci = groups(pp)
        a.errorbar(xp, ob, yerr=ci, fmt=mk + "-", ms=4, capsize=2,
                   lw=1.2, label=lab,
                   color=COL[nm] if lab == "Original" else "grey")
    a.plot([0, 0.3], [0, 0.3], "k--", lw=0.8)
    a.set_xlim(0, 0.3); a.set_ylim(0, 0.3)
    a.set_title(NAME[nm], fontsize=10.5)
    a.set_xlabel("Predicted probability")
    a.set_ylabel("Observed proportion")
    a.text(0.012, 0.28,
           f"AUROC {roc_auc_score(y, p):.2f}\n"
```

```
            f"Intercept (CITL) {m0.params[0]:.2f}\n"
            f"Slope {m1.params[1]:.2f}",
            va="top", fontsize=8.5)
    a.legend(loc="lower right", fontsize=8)
plt.tight_layout()
plt.savefig("Figure2_calibration.png", dpi=300)
plt.show()

# ---------- Figure 3: decision curves ----------
pts = np.arange(0.01, 0.2001, 0.005)
def nb(p):
    out = []
    for t in pts:
        tp = ((p >= t) & (y == 1)).sum()
        fp = ((p >= t) & (y == 0)).sum()
        out.append(tp / n - fp / n * t / (1 - t))
    return np.array(out)

fig, axs = plt.subplots(1, 2, figsize=(11, 4.5), sharey=True)
for j, rec in enumerate([False, True]):
    a = axs[j]
    a.plot(pts, nb(np.ones(n)), color="black", lw=1.2, ls="--",
           zorder=5, label="Treat all")
    a.axhline(0, color="black", lw=1, label="Treat none")
    for nm in X:
        lp = X[nm].values
        if rec:
            m1 = sm.GLM(y, sm.add_constant(lp), family=Bn).fit()
            p = m1.predict(sm.add_constant(lp))
        else:
            p = 1 / (1 + np.exp(-lp))
        a.plot(pts, nb(p), lw=1.6, color=COL[nm], label=NAME[nm])
    a.set_ylim(-0.01, 0.035)
    a.set_xlabel("Threshold probability")
    a.set_title("A. Original models" if not rec
                else "B. After recalibration (intercept and slope)",
                fontsize=10.5)
    if j == 0:
        a.set_ylabel("Net benefit")
    a.legend(fontsize=8)
plt.tight_layout()
plt.savefig("Figure3_decision_curves.png", dpi=300)
plt.show()
print("Сохранено: Figure1_flow.png, Figure2_calibration.png,"
      " Figure3_decision_curves.png")
```